

Bidot - - Orignac
Tim
G1A

SAE31 - Project FluxMedia

Séance n°1 (NE) : Introducing GSTREAMER

Un premier exemple « basique »

Commençons avec le pipeline le plus court que l'on puisse imaginer : émettre un son pur avec le greffon `audiotestsrc` et l'écouter avec `autoaudiosink`, qui envoie le flux vers les haut-parleurs de votre machine. Simple. Exécutez `gst-launch-1.0 audiotestsrc ! autoaudiosink` ! Pour avoir des informations sur un greffon, vous pouvez utiliser la fonction `gst-inspect-1.0`. Exécutez la ligne de commande `gst-inspect-1.0 audiotestsrc`. Observez les informations qui s'affichent.

Réponse :

commandes exécutez :

```
gst-launch-1.0 audiotestsrc ! autoaudiosink
```

observation :

Quand nous exécutons la commande, nous entendons un bip continu durant 99:99:99 dans les haut-parleurs. C'est le son de test par défaut de GStreamer : une hône sinusoïdale de 440 Hz

commandes exécutez :

```
gst-inspect-1.0 audiotestsrc
```

observation :

La commande affiche toutes les informations du greffon `audiotestsrc`. Nous pouvons observer que ce greffon sert à créer des signaux audio de test

Les propriétés des greffons

Les greffons possèdent de nombreuses propriétés (`element properties`), que l'on peut utiliser pour personnaliser les greffons. On les positionne juste à la suite du nom du greffon sans « ! », selon la syntaxe suivante : `gst-launch-1.0 greffon_1 elemProp1=param1 ! greffon_2`. Dans cet exemple, on personnalise la propriété `elemProp1` du greffon `_1` avec la variable `param1`. Exécutez la ligne de commande `gst-launch-1.0 audiotestsrc freq=880 ! autoaudiosink`. Ecoutez le résultat, et expliquez.

Réponse :

commandes exécutez :

```
gst-launch-1.0 audiotestsrc freq=880 ! autoaudiosink.
```

observation :

Nous entendons un son plus aigu que dans le premier test.
En ajoutant le paramètre freq=880, nous avons pu doubler la fréquence du signal (de 440 Hz à 880 Hz)

Modifiez la ligne de commande ci-dessus pour produire un bruit bleu, avec un volume à 50%.

Réponse :

commandes exécutez :

```
gst-launch-1.0 audiotestsrc wave=11 volume=0.5 ! autoaudiosink
```

observation :

Cette commande génère un bruit bleu grâce au paramètre wave=11. Ceci nous donne un bruit aléatoire contenant surtout des fréquences aiguës
Le paramètre volume=0.5 règle le volume à 50 % donc le son est moins fort que précédemment

Découvrir les flux

gst-discoverer-1.0 est une autre fonction très utile pour obtenir des informations sur les flux et leurs métadonnées. Téléchargez le fichier jecpakic.oops sur moodle (fichiers utiles TP n°1). Exécutez gst-discoverer-1.0 jecpakic.oops. Est-ce un fichier audio ou vidéo ? Justifiez. Indiquez ci-dessous le format, la durée, la fréquence d'échantillonnage, le nombre de bits par échantillon, et le nombre de pistes contenu dans ce fichier.

Réponse :

commandes exécutez :

```
gst-discoverer-1.0 jecpakic.oops
```

observation :

Le fichier jecpakic.oops est un **fichier audio** non compressé. Format : audio/x-wav, 44100 Hz, 16 bits, 2 canaux (stéréo). Il ne contient aucune piste vidéo.
Nous avons obtenu ces informations grâce à la commande gst-discoverer-1.0, qui permet d'afficher les métadonnées d'un flux multimédia.

```
minimoys@MacBook-Pro-2 Downloads % gst-discoverer-1.0 jecpakic.oops

Analyzing file:///Users/minimoys/Downloads/jecpakic.oops
Done discovering file:///Users/minimoys/Downloads/jecpakic.oops

Properties:
  Duration: 0:00:00.500000000
  Seekable: yes
  Live: no
  audio #0: WAV
    Stream ID: 5c3092e38beadbe313da439cfe08687bce2bcbf997956891906a2b61d4abd17d
    Language: <unknown>
    Channels: 1 (unknown layout)
    Sample rate: 44100
    Depth: 16
    Bitrate: 705600
    Max bitrate: 0

minimoys@MacBook-Pro-2 Downloads % █
```

Format : WAV (audio non compressé)

Durée : 0,5 seconde

Fréquence d'échantillonnage : 44 100 Hz

Nombre de bits par échantillon : 16 bits

Nombre de pistes : 1 (mono)

« Nouvelle vague »

Nous allons maintenant mettre en place un pipeline pour écouter un fichier non compressé, encodé au format wave (WAV). Le pipeline est le suivant : L'étape d'analyse permet de connaître la structure globale du fichier. Cherchez un greffon permettant de lire un fichier source stocké sur votre disque dur. Inspectez-le et trouvez la propriété qui permet d'indiquer le chemin du fichier à lire.

Réponses :

commandes exécutez :

```
gst-inspect-1.0 | grep -i filesrc
gst-inspect-1.0 filesrc
```

observation :

En inspectant le greffon filesrc, nous avons vu qu'il sert à lire un fichier local depuis le disque dur. Sa propriété principale est location, qui permet d'indiquer le chemin complet du fichier à lire

```
minimoys@MacBook-Pro-2 Downloads % gst-inspect-1.0 | grep -i filesrc

coreelements: filesrc: File Source
debug: pushfilesrc: Push File Source
multifile: multifilesrc: Multi-File Source
multifile: splitfilesrc: Split-File Source
rsfile: rsfilesrc: File Source
minimoys@MacBook-Pro-2 Downloads % gst-inspect-1.0 filesrc

Factory Details:
Rank                primary (256)
Long-name           File Source
Klass               Source/File
Description         Read from arbitrary point in a file
Author             Erik Walthinsen <omega@cse.ogi.edu>
Documentation       https://gstreamer.freedesktop.org/documentation/coreelements/filesrc.html

Plugin Details:
Name               coreelements
Description        GStreamer core elements
Filename          /opt/homebrew/Cellar/gstreamer/1.26.8/lib/gstreamer-1.0/libgstcoreelements.dylib
Version           1.26.8
License           LGPL
Source module     gstreamer
Documentation     https://gstreamer.freedesktop.org/documentation/coreelements/
Source release date 2025-11-10
Binary package    GStreamer source release
Origin URL        https://github.com/Homebrew/homebrew-core

GObject
+----GInitiallyUnowned
+----GstObject
+----GstElement
+----GstBaseSrc
+----GstFileSrc

interfaces implémentées :
GstURISHandler

Element Flags:
- SOURCE

Pad Templates:
SRC template: 'src'
Availability: Always
Capabilities:
ANY

Element has no clocking capabilities.

URI handling capabilities:
Element can act as source.
Supported URI protocols:
file

Pads:
SRC: 'src'
Pad Template: 'src'

Element Properties:
```

Cherchez un greffon permettant d'analyser (parse en anglais) un fichier WAV.

Réponse :

commandes exécutez :

```
gst-inspect-1.0 | grep -i wav
```

```
gst-inspect-1.0 wavparse
```

observation :

Le greffon utilisé pour analyser un fichier WAV est wavparse, permet de lire la structure du fichier et d'en extraire le flux audio brut.

```
minimoys@MacBook-Pro-2 ~ % gst-inspect-1.0 | grep -i wav

audioparsers: wavpackparse: Wavpack audio stream parser
audiovisualizers: wavescopes: Waveform oscilloscope
libav: avdec_adpcm_ima_wav: libav ADPCM IMA WAV decoder
libav: avdec_adpcm_swf: libav ADPCM Shockwave Flash decoder
libav: avenc_adpcm_ima_wav: libav ADPCM IMA WAV encoder
libav: avenc_adpcm_swf: libav ADPCM Shockwave Flash encoder
libav: avmux_avm2: libav SWF (ShockWave Flash) (AVM2) muxer
libav: avmux_swf: libav SWF (ShockWave Flash) muxer
libav: avmux_w64: libav Sony Wave64 muxer
libav: avmux_wav: libav WAV / WAVE (Waveform Audio) muxer (not recommended, use wavenc instead)
typefindfunctions: application/x-shockwave-flash: swf, swfl
typefindfunctions: audio/x-wav: wav
typefindfunctions: audio/x-wavpack: wv, wvp
typefindfunctions: audio/x-wavpack-correction: wvc
wavenc: wavenc: WAV audio muxer
wavparse: wavparse: WAV audio demuxer
minimoys@MacBook-Pro-2 ~ % gst-inspect-1.0 wavparse

Factory Details:
Rank                primary (256)
Long-name           WAV audio demuxer
Klass               Codec/Demuxer/Audio
Description         Parse a .wav file into raw audio
Author             Erik Walthinsen <omega@cse.ogi.edu>
Documentation       https://gstreamer.freedesktop.org/documentation/wavparse/#wavparse-page

Plugin Details:
Name               wavparse
Description        Parse a .wav file into raw audio
Filename          /opt/homebrew/Cellar/gstreamer/1.26.8/lib/gstreamer-1.0/libgstwavparse.dylib
Version           1.26.8
License           LGPL
Source module     gst-plugins-good
Documentation     https://gstreamer.freedesktop.org/documentation/wavparse/
Source release date 2025-11-10
Binary package    GStreamer Good Plug-ins source release
Origin URL        https://github.com/Homebrew/homebrew-core

GObject
+----GInitiallyUnowned
+----GstObject
+----GstElement
+----GstWavParse

Element Flags:

Pad Templates:
SINK template: 'sink'
Availability: Always
Capabilities:
audio/x-wav
audio/x-rf64

SRC template: 'src'
Availability: Always
Capabilities:
audio/ms-gsm
audio/mpeg
mpegversion: 1
```

Proposez la ligne de commande complète pour écouter le fichier MfN.wav (à télécharger sur moodle > fichiers utiles TP n°1).

Réponse :

commandes exécutez :

```
gst-launch-1.0 filesrc location=MfN.wav ! wavparse ! autoaudiosink
```

observation :

Cette commande va lire le fichier WAV MfN.wav.

filesrc lit le fichier sur le disque,

wavparse analyse sa structure audio,

autoaudiosink envoie le son vers les haut-parleurs.

```
Liberation du pipeline...
minimoys@MacBook-Pro-2 ~ % cd Downloads
minimoys@MacBook-Pro-2 Downloads % gst-launch-1.0 filesrc location=MfN.wav ! wavparse ! autoaudiosink
Définition du pipeline à PAUSED...
Le pipeline est en phase de PREROLL...
Redistribution de latence...
Le pipeline a terminé la phase PREROLL...
Passage du pipeline à la phase PLAYING...
Redistribution de latence...
New clock: GstAudioSinkClock
0:00:17.4 / 0:01:00.0 (29,1 %)
```

Et sauter dans les flac...»

Nous allons maintenant mettre en place un pipeline pour écouter un fichier compressé (sans pertes) au format FLAC. Le pipeline pour écouter un fichier audio compressé est le suivant :

Quel greffon pour parser un fichier flac ?

commandes exécutez :

```
gst-inspect-1.0 | grep -i flac
```

observation :

```
minimoys@MacBook-Pro-2 Downloads % gst-inspect-1.0 | grep -i flac
audioparsers:  flacparse: FLAC audio parser
claxon:  claxondec: Claxon FLAC decoder
flac:  flacdec: FLAC audio decoder
flac:  flacenc: FLAC audio encoder
flac:  flactag: FLAC tagger
libav:  avdec_flac: libav FLAC (Free Lossless Audio Codec) decoder
typefindfunctions: application/x-id3v1: mp3, mp2, mp1, mpga, ogg, flac, tta
typefindfunctions: application/x-id3v2: mp3, mp2, mp1, mpga, ogg, flac, tta
typefindfunctions: audio/x-flac: flac
minimoys@MacBook-Pro-2 Downloads %
```

Quel greffon pour décoder un fichier flac ?

commandes exécutez :

```
gst-inspect-1.0 | grep -i flacdec
```

observation :

```
minimoys@MacBook-Pro-2 Downloads % gst-inspect-1.0 | grep -i flacdec
flac: flacdec: FLAC audio decoder
minimoys@MacBook-Pro-2 Downloads %
```

Proposez la ligne de commande complète pour écouter le fichier AJS_HD_short.flac (à télécharger sur moodle > fichiers utiles TP n°1).

commandes exécutez :

```
gst-launch-1.0 filesrc location=AJS_HD_short.flac ! flacparse ! flacdec ! autoaudiosink
```

observation :

Cette commande permet d'écouter un fichier audio FLAC

```
minimoys@MacBook-Pro-2 Downloads % gst-launch-1.0 filesrc location=AJS_HD_short.flac ! flacparse ! flacdec ! autoaudiosink
Définition du pipeline à PAUSED...
Le pipeline est en phase de PREROLL...
Redistribution de latence...
Redistribution de latence...
Le pipeline a terminé la phase PREROLL...
Passage du pipeline à la phase PLAYING...
Redistribution de latence...
New clock: GstAudioSinkClock
0:00:13.2 / 0:00:22.3 (59,5 %)
```

Afficher une image

GSTREAMER permet aussi de manipuler des vidéos. Commençons par les bases : afficher une image fixe, compressée au format JPEG. Proposez la ligne de commande complète pour afficher le fichier image.jpeg (à télécharger sur moodle > fichiers utiles TP n°1). Pensez que Gstreamer affiche des flux : pour afficher une seule image, il faut donc préciser dans le pipeline que l'on veut « figer » l'image avant affichage.

Réponse :

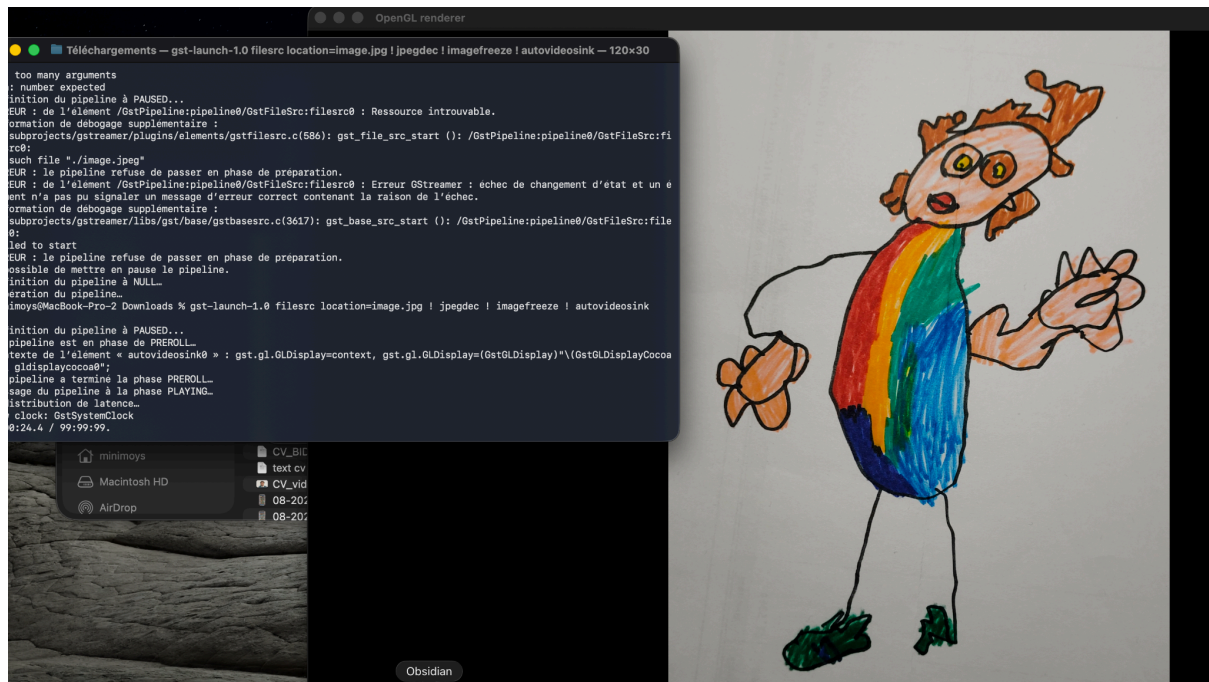
commandes exécutez :

```
gst-launch-1.0 filesrc location=image.jpg ! jpegdec ! imagefreeze ! autovideosink
```

observation :

Cette commande permet d'afficher une image JPEG:

filesrc lit le fichier image sur le disque,
jpegdec décode l'image compressée au format JPEG,
imagefreeze transforme l'image décodée en flux vidéo d'une seule image
autovideosink affiche l'image dans une fenêtre à l'écran.



« This is the end... »

Pour terminer cette initiation, déterminez la ligne de commande permettant d'écouter le fichier sample.mp3 (à télécharger sur moodle > fichiers utiles TP n°1).

Réponse :

commandes exécutez :

gst-launch-1.0 filesrc location=sample.mp3 ! decodebin ! autoaudiosink

observation :

filesrc lit le fichier MP3 sur le disque,
decodebin détecte automatiquement le bon décodeur pour décoder le flux audio MP3,
autoaudiosink envoie ensuite le son vers la sortie audio (haut-parleurs).

```
minimoys@MacBook-Pro-2 ~ % cd Downloads
minimoys@MacBook-Pro-2 Downloads % gst-launch-1.0 filesrc location=sample.mp3 ! decodebin ! autoaudiosink

Définition du pipeline à PAUSED...
Le pipeline est en phase de PREROLL...
Redistribution de latence...
Le pipeline a terminé la phase PREROLL...
Passage du pipeline à la phase PLAYING...
Redistribution de latence...
New clock: GstAudioSinkClock
0:00:17.6 / 0:00:19.1 (92,1 %)
```

